Advanced text embeddings- Doc2Vec and GloVe

by Dr. NiB
@NITKKR 2026

Word embeddings (Quick recap)

- What?
  - **Vectors of numbers** (0-1 or real ? weights?)
- Why?
  - So that we can apply ML on text data (by letting the **machine understand our text**)
- How?
  - **Count or frequency based** (OHE,BOW,TF-IDF) :*nltk CountVectorizer (used in lab?) is BOW*
  - **Deep learning trained models** (Word2Vec,GLoVe,doc2Vec)

- **Prerequisites (Covered):** BoW, TF-IDF, Word2Vec
- **To do: Doc2Vec and GloVe which are Deep Learning Model Based Embeddings i.e. optimised neural net training weights become embeddings**

Recap

NN Inputs (Starting State)	how to create inputs for each unique word in the training data for the Neural Network.
Random Weights	connection **Weights** start out with **random values**.
Optimization via Backpropagation	use of **Backpropagation** to optimize these weights after the NN makes predictions.
Prediction using SoftMax	The NN connects activation functions to outputs, running them through the **SoftMax function** because there are multiple outputs for classification. The loss function used for Backpropagation is **Cross Entropy**.
Result: Word Embeddings	The final embedding is derived from the optimized weights: **"The new Weights on the connections from the inputs to the Activation Functions are the Word Embeddings."**.
Word2Vec Variants	flow of **Continuous Bag of Words (CBoW)**, which predicts the middle word using surrounding words, and **Skip-Gram**, which predicts surrounding words using the middle word.

I like nlp CBOW vs skip gram

Window size = 2

I like nlp

Nlp:i = 1

Co occurence

I like

[D [P] [ ] [[S] [[W][][]]]] DENSE EMBEDDINGS:

Count vs. Deep Learning Embeddings

word->sentence->paragraph->document

Concept	**Count Vectorizer (OHE, BoW, TF-IDF)**	**Word Embeddings (Word2Vec, GloVe, Doc2Vec)**
What?	Vectors of numbers (0 or 1, or frequencies/weights).	**Dense vectors** (continuous values/floats).
Representation	**Sparse** (many zeros), leads to heavy computation.	**Dense** (no zeros), addresses sparsity.
Dimensionality	**High:** Defined by vocabulary size (e.g., 15k, 100k+).	**Low:** Hyperparameter (e.g., 50, 100, 300).
Context/Semantics	**Limited:** Order is ignored (Bag of Words loss).	**Capture semantic relationships/contextual nuances**.

Word embedding /DEEP LEARNING BASED EMBEDDINGS addresses the problem of sparsity and high dimensionality found in BoW/Count methods.

Motivation: Why GloVe and Doc2Vec?

Limitations of Word2Vec

• **Local Context Only:** Word2Vec (Skip-Gram/CBoW) learns context primarily from **neighboring words** within a window size.
• **Ignoring Global Statistics:** It does not inherently leverage overall word co-occurrence statistics across the entire corpus, which GloVe handles.
• **No Document Vector:** Standard Word2Vec provides word embeddings, but we need a single vector for an entire document or article for tasks like **Clustering** or **Classification**.

Solution Paths (Deep Learning Model Based Embeddings)

1. **GloVe (Global Vectors):** Incorporates Global Co-occurrence Statistics.
2. **Doc2Vec:** Extends Word2Vec to generate vectors for entire documents (Paragraph Vectors)

GloVe

What is GloVe? •Global Vectors for Word Representation

•How does it work?
Combines advantages of local context models (like Word2Vec) with global matrix factorization to capture context across the **entire corpus**.

• **Pre-trained Models:** GloVe models are often pre-trained on massive corpora, such as **Wikipedia 2014** and **Gigaword 5**.

• **Scale:** These pre-trained models can involve 6 Billion tokens and 400,000 vocabulary words.

https://nlp.stanford.edu/pubs/glove.pdf

How are the dense vectors actually created? Word embeddings are derived by training a **Neural Network (NN)**.

Learning Embeddings (Recalling Word2Vec/NN Mechanics)

1. **Input:** Create inputs for **each unique word**.
2. **Initial State: Weights** on the network connections start with **random values**.
3. **Prediction:** The network uses an **Activation Function** (and SoftMax for classification) to make predictions (e.g., predicting the surrounding word (Skip-Gram) or the word in the middle (CBoW)).
4. **Optimization:** Weights are iteratively adjusted using a loss function (like **Cross Entropy**) and **Backpropagation**.
5. **Result:** The **new Weights** on the connections from the inputs to the **Activation Functions** *are* the **Word Embeddings**.

Vector Arithmetic

Since embeddings capture semantic relationships, they allow for meaningful vector calculations

Operation	Interpretation	Result (Similarity Score)
V(King) - V(Man) + V(Woman)	Finding the woman equivalent of King.	High similarity (e.g., **73%**) to **Queen**.
V(Man) - V(Male) + V(Female)	Subtracting male gender and adding female gender to species ‘man'.	Top result is **Woman**.
V(God) + V(Cricket) + V(India)	Querying the context of the "God of Cricket in India."	Top result is **Sachin**.
V(Dog) + V(Newborn)	Finding the name of the newborn of a dog.	Top results :**Puppy** (80%) ,**Pup** (76%).

The word vectors are so **contextually aware** that simple addition/subtraction reveals complex analogies.

Doc2Vec Goal

• Doc2Vec (also related to Paragraph Vectors) aims to generate a single continuous vector representation for an entire document.

Doc2Vec/Document Vector Applications

Task	Doc2Vec's role
Clustering	Grouping documents based on semantic similarity.
Classification	Tasks like sentiment analysis (predicting positive/negative).
Recommendation	Suggesting similar documents or articles.
Dimensional Reduction	Providing a low-dimensional feature representation for input data.
Search/RAG	Modern applications involving Retrieval Augmented Generation.
Anomaly Detection	Finding outliers.

Document Vectorization

The most common conceptual approach to obtain a document vector using pre-trained word embeddings is averaging:

1. **Process:** Each document is first tokenized into words.
2. **Vector Collection:** The word vector (e.g., 300-dimensional GloVe vector) is fetched for every word in the document.
3. **Averaging:** The final **Document Vector** is calculated as the **mean (average)** of all the individual word vectors collected for that document.
   - *Example:* If a document yields 11 word vectors (each 1×300), the document vector is the average of those 11 vectors, resulting in one 1×300 dense vector.

Result: The final dense vector matrix has rows equal to the number of documents and columns equal to the embedding size (e.g., 300 dimensions)

If you train a Word2Vec model on a small corpus of 1,000 documents, should you use a high embedding dimension (e.g., 300) or a low dimension (e.g., 20)?

A: Low dimension (e.g., 10 or 20). High dimensionality (e.g., 200 or 300) is typically reserved for very large corpora (millions of documents) because the model needs to learn that many contexts and meanings

Limitations

Limitation	Embeddings like Word2Vec and GloVe are considered **static embeddings**
Polysemy	A single word has only one vector, regardless of context (e.g., "bank").
Context Mismatch	Semantically similar words may be far apart if they appear in different contexts (e.g., `iii`, `third`, and `three` appear in different text contexts and are not close in the projected space).
Data Bias	Visualization reveals biases in the training data, such as strong coverage of US geography but poor coverage of other regions (e.g., nearby neighbors of `boston` are other US cities).
OOV Words	Cannot generate vectors for words not present in the pre-trained vocabulary.

[TensorFlow's Embedding Projector](#) flattens static vectors (like those trained on Google News articles up to 2013) into a **3D space** to analyze these properties

Deep learning (neural net based) models for NLP

Coverage

- Convolutional Neural Network (CNN),
- Recurrent Neural Network (RNN),
- Long Short-Term Memory (LSTM) ,
- Gated Recurrent Unit(GRU) using Natural Language Toolkit (**NLTK**)

CNN (Convolutional Neural Network)

Used for: *Text classification, sentence modeling, keyword detection.*

- Think of CNNs as *feature detectors*: they learn patterns like **n-grams**, independent of position.
- **Convolution filters** slide over word embeddings (like 3-word windows), capturing local features.
- **Pooling** (usually max-pooling) keeps the strongest signal → feeds into dense layer → classification.

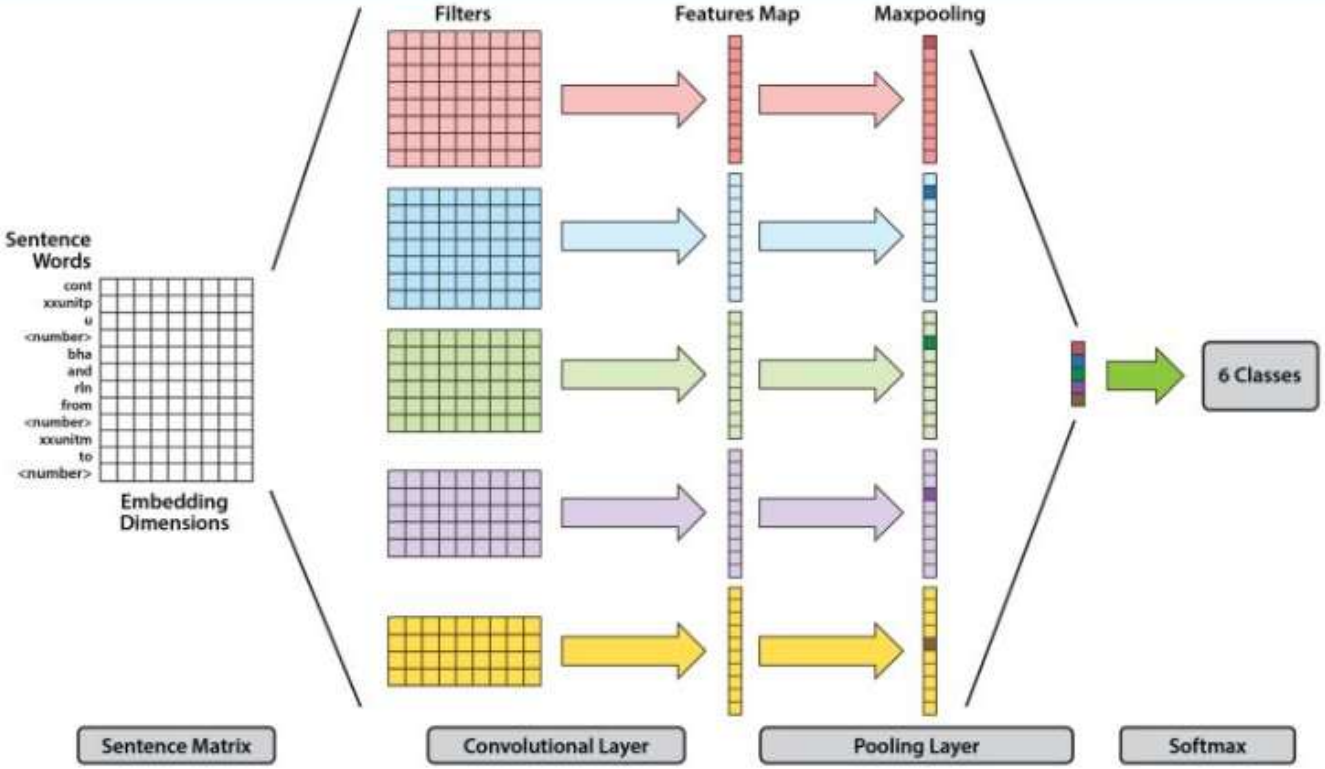
In NLP: Great for tasks like **sentiment analysis, topic classification, question type detection**.

[I | love | NLP | a lot]

↓ ↓ ↓ ↓

Embeddings → 1D filters → Feature maps → Max-pool → Dense → Output

CONVOLUTIONAL NEURAL NETWORK-BASED TEXT CLASSIFICATION NETWORK



Given a sentence, a convolutional neural network uses convolutional layers to refine representations of input words, before combining them to render a classification.

https://www.deeplearning.ai/resources/natural-language-processing/

RNN (Recurrent Neural Network)

Used for: *Sequential modeling — word-by-word dependency.*

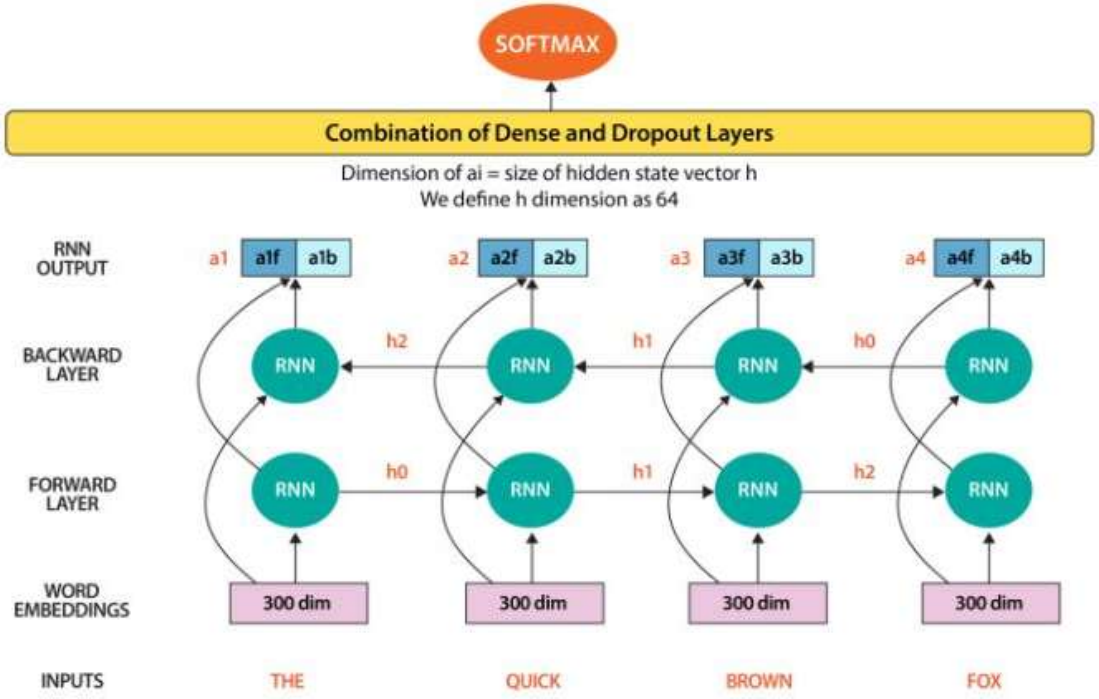
- Unlike CNNs, RNNs handle **order and context** by maintaining a **hidden state** that flows through time.
- But they suffer from **vanishing/exploding gradients** → hard to remember long-term dependencies.

In NLP: Used in **language modeling, text generation, translation** (early seq2seq).

x1 → [h1] → [h2] → [h3] → [h4] → Output

↑ (hidden state carries context)

RECURRENT NEURAL NETWORK



A bidirectional recurrent neural network processes the input both forward and backward to improve the representations it produces.

RNNs have been used to generate mathematical proofs and translate human thoughts into words.

https://www.deeplearning.ai/resources/natural-language-processing/

LSTM (Long Short-Term Memory)

Used for: *Capturing long-term dependencies.*

- Improves RNN by adding **gates** that control information flow:
  - **Forget gate** → what to drop
  - **Input gate** → what to add
  - **Output gate** → what to show
- Maintains **cell state** (long-term memory) + **hidden state** (short-term output).

```
x_t → │ Input → Forget → Output gates│ → h_t
      └──────────→ Cell state c_t ──────┘
```

In NLP: Used in **machine translation, speech recognition, sentiment analysis.**

GRU (Gated Recurrent Unit)

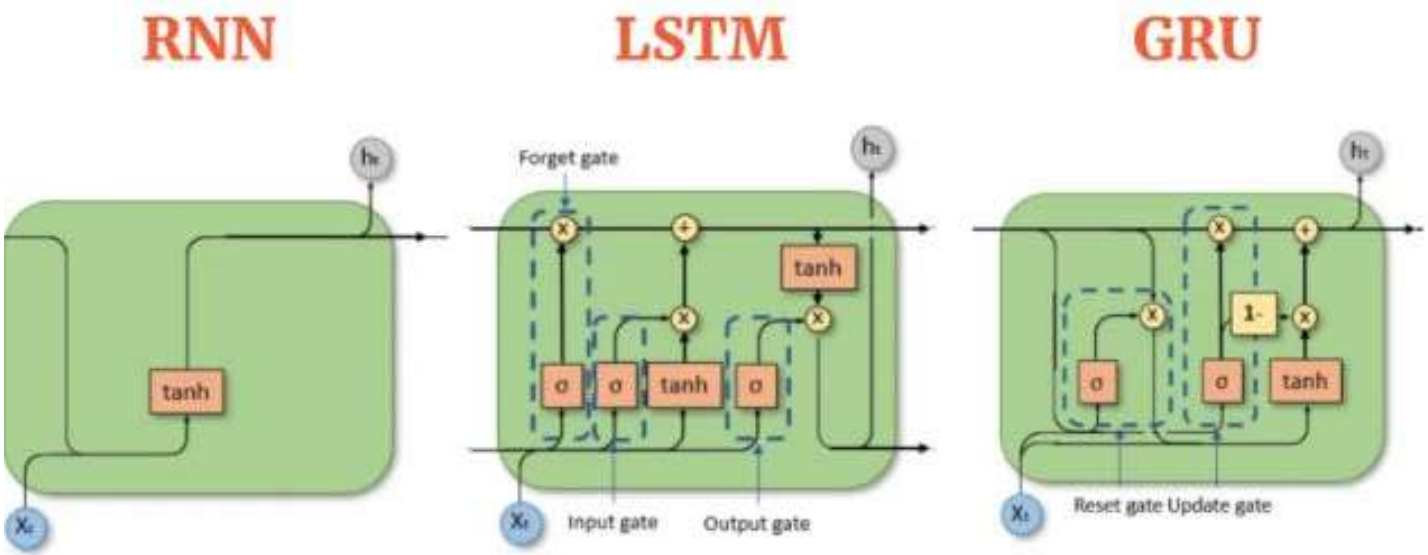
Used for: *Simpler, faster alternative to LSTM.*

- Combines forget + input gates into **update gate**; has no separate cell state.
- Fewer parameters → trains faster, similar performance.

$x_t + h_{(t-1)} \rightarrow$ [Update gate | Reset gate] $\rightarrow h_t$

In NLP: Used for **sequence labeling, chatbots, text summarization**.

Sequence Models Compared: RNNs, LSTMs, GRUs



Description	Recurrent Neural Network (RNN)	Long Short Term Memory (LSTM)	Gated Recurrent Unit (GRU)
Overview	RNNs are foundational sequence models that process sequences iteratively, using the output from the previous step as an input to the current step.	LSTMs are an enhancement over standard RNNs, designed to better capture long-term dependencies in sequences.	GRUs are a variation of LSTMs with a simplified gating mechanism.
Use Cases	Due to its limitations, plain RNNs are less common in modern applications. Used in simple language modeling, time series prediction	Machine translation, speech recognition, sentiment analysis, and other tasks that require understanding of longer context.	Text generation, sentiment analysis, and other sequence tasks where model efficiency is a priority.
Model variants	Vanilla RNN, Bidirectional RNN, Deep (Stacked) RNN	Vanilla LSTM, Bidirectional LSTM, Peephole LSTM, Deep (Stacked) LSTM	GRU

Credits: aiml.com

Description	Recurrent Neural Network (RNN)	Long Short Term Memory (LSTM)	Gated Recurrent Unit (GRU)
Key characteristics	- Recurrent connections allow for the retention of "memory" from previous time steps.	- Uses gates (input, forget, and output) to regulate the flow of information. - Has a cell state in addition to the hidden state to carry information across long sequences.	- Contains two gates: reset gate and update gate. - Merges the cell state and hidden state.
Advantages	- Simple structure. - Suitable for tasks with short sequences.	- Can capture and remember long-term dependencies in data. - Mitigates the vanishing gradient problem of RNNs.	- Fewer parameters than LSTM, often leading to faster training times. - Simplified structure while retaining the ability to capture long-term dependencies.
Disadvantages	- Suffers from the vanishing and exploding gradient problem, making it hard to learn long-term dependencies - Limited memory span	- More computationally intensive than RNNs - Complexity can lead to longer training times.	- Might not capture long-term dependencies as effectively as LSTM in some tasks.

Summary

- Though CNNs capture neighbourhood via window/n-grams, so New Delhi is one instance for them, rather than two tokens New and Delhi, CNNs do not appreciate sequential structure of language.

- RNNs acknowledge the dependence of a word on the previous word and remember this context via hidden states, while connecting it to the current task.

- The architectures known as Gated Recurrent Unit (GRU) and long short-term memory (LSTM) are types of RNNs designed to remember information for an extended period.

- GRU keeps contextual information in both directions, which is helpful in text classification.